

ZENITH

Software Supply Chain Risk Scorer

SBOM Analyzer — Problem Statement PB-10

Platform Architecture · Analysis Algorithms · User Interface Design

Team Zenith · Societe Generale iHackMyPlace Hackathon ·

July 2026

LINKS:

Git Repository: https://github.com/Bhavesht-Reddy/Zenith_SG.git

Solution Video:

https://drive.google.com/drive/folders/1C8VnIQv7zyZuChMI07SFUUpLhvYP6Ei?usp=drive_link

1. Executive Summary

Zenith is a software supply chain risk scoring platform built for the Societe Generale iHackMyPlace hackathon (PB-10). It ingests an SBOM, a vulnerability database, a license-rule table, and a transitive-dependency graph; resolves every dependency path from each application to each flagged library; scores and ranks risk per application; and presents results in an offline-capable dashboard with plain-English remediation narratives.

The platform uses a hybrid, deterministic-core-first architecture. The core is the product — it runs, passes self-eval, and produces the full ranked report with ML and LLM layers switched off. The ML layer only re-orders prioritization; the LLM layer only rewrites already-decided findings into prose. Removing either degrades nothing about correctness.

Metric	Target	Result	Status
Vulnerability Detection	> 85%	98.3% (173/176)	PASS
Transitive Resolution	100%	100% (150/150)	PASS
License Conflict Detection	> 90%	100% (16/16)	PASS
False Positive Rate	< 20%	39.1%*	MISS*
Risk Score Accuracy (± 10)	> 90%	68.0%*	MISS*

* The two misses are dataset-inherent: the DB's affected_versions ranges provably contradict the ground-truth labels. The team chose maximum security value (catch every name-match) over gaming FPR by suppressing true findings. See §3.4 for details.

2. Platform Architecture

2.1 Three-Layer Architecture

The system has three additive layers. The deterministic core is authoritative; the ML and LLM layers are strictly additive and non-blocking.

Layer	Function	Dependencies	Removable?
1 — Deterministic Core	Graph construction, path enumeration, CVE matching, license engine, findings	Python stdlib + NetworkX	No (IS the product)
2 — ML Risk Scorer	Numeric 0-100 risk score per dependency (12 features)	scikit-learn, NumPy, pandas	Yes
3 — LLM Narratives	Plain-English remediation for top-N findings	Ollama API / template fallback	Yes

2.2 Data Flow Pipeline

Stage	Command	Output
1. Ingest	python -m src.ingestion	Validated Dataset object (6 files)
2. Findings	python -m src.findings	raw_findings.json + classifications.json
3. Score	python -m src.ml.train	risk_scores.json + ml_metrics.json
4. Report	python -m src.report.composite	risk_report.json (ranked per-app)
5. Narrate	python -m src.llm.narrative	narratives_cache.json (optional)
6. Serve	python -m dashboard.app	Dashboard at :5000 + HTML/PDF export

2.3 Layer 1 — Deterministic Core

The core comprises four subsystems, all stdlib-only (except NetworkX for graphs):

- **Graph (src/graph/):** Builds a NetworkX DiGraph with per-app subgraph isolation. Nodes: ("app", app_id) and ("lib", app_id, lib_name). Edges wired by library name (not version — the edge file's versions are inconsistent with the SBOM). 10 orphan transitive rows are attached directly to the app root.

- **Path Enumeration (traversal.py):** Uses all_simple_paths (DFS) to find every distinct app→library chain. Multiple paths (diamonds) are all returned — compound risk is never deduplicated. This backs the 100% transitive resolution result.
- **CVE Matching (vuln_match.py):** Library-name match raises a finding; version-in-range is a confidence tier, not a filter. Supports exact strings, comparators (<=, >=, <, >, ==), and range pairs (with auto-normalization of 39 reversed entries).
- **License Engine (license/engine.py):** Rule-table driven with distribution-scope awareness. Viral copyleft in a proprietary app → CONFLICT; same in internal-only → ADVISORY (not distributed); unknown license → UNKNOWN (never silently assumed compatible).

2.4 Layer 2 — ML Risk Scorer

12 engineered features per dependency (CVSS, patch flag, depth, staleness, license risk, path count, shared-across-apps, criticality). Two scorers evaluated on an honest 80/20 stratified holdout (seed 42):

Scorer	Precision	Recall	F1	FPR
Baseline formula (hand-tuned)	73.4%	100%	0.847	32.1%
GBDT (HistGBC)	83.3%	74.5%	0.786	13.2%

The GBDT loses on honest holdout (F1 0.786 vs 0.847) and drops 25 points of recall. Per project policy — if neither clearly wins, ship the simpler formula — the transparent baseline is shipped.

2.5 Layer 3 — LLM Narrative Layer

Generates remediation narratives for the top-N findings (default 10). The prompt contains structured data only. Every narrative is cached to narratives_cache.json as produced (crash-safe, cache-first). Two backends: Ollama (local, open-source) → deterministic template fallback. A quality gate rejects responses that echo structured input. No logic depends on a specific model identity.

2.6 Technology Stack & Deployment

Component	Technology	Purpose
Web Framework	Flask 3.1.3	Dashboard and API routes
ML Framework	scikit-learn 1.7.2	HistGradientBoostingClassifier
Graph Library	NetworkX	Dependency graph and path enumeration
Deployment	Render (Gunicorn)	Production WSGI server
Data Processing	NumPy 1.26.4, Pandas 2.3.2	Feature computation

3. Analysis Algorithms

3.1 Data Ingestion & Validation

The ingestion module loads six files with strict schema enforcement: required-field presence checks, cross-reference integrity (application_id foreign keys, dep_id alignment), boolean normalization, cp1252 encoding auto-detection, and risk_level standardization.

File	Format	Key Fields
applications.json	JSON	app_id, name, criticality, license_model
sbom_dependencies.csv	CSV	dep_id, application_id, library, version, license, dependency_type, last_updated
vulnerability_db.json	JSON	cve_id, library, affected_versions, cvss_score, patch_available
license_rules.json	JSON	license, risk_level, compatible_with_proprietary, viral
dependency_labels.csv	CSV (cp1252)	dep_id, is_risky, risk_type, severity

transitive_dependencies.js on	JSON	parent_library, child_library, application_id
----------------------------------	------	---

3.2 Graph Construction & Path Enumeration

The graph builder creates a NetworkX DiGraph with per-application isolation using tuple-based node keys: ("app", app_id) for application roots and ("lib", app_id, lib_name) for libraries. This prevents cross-application path leakage through shared libraries.

Construction proceeds in three phases: (A) Create all app and library nodes from the SBOM. (B) Wire edges — direct deps connect to app roots; transitive edges from the edge file are wired by library name within each app (version fields ignored due to inconsistencies). (C) Attach 10 orphan transitive rows directly to the app root (orphan=True flag) so no dependency is ever pathless.

Path enumeration uses NetworkX all_simple_paths (DFS). The graph is a DAG by construction, so enumeration terminates and finds every distinct path. Diamond dependencies (multiple paths to the same library) are all returned — compound risk is never deduplicated. The depth_of() function provides shortest hop count as an ML feature.

3.3 Vulnerability Matching

A pre-built vulnerability index groups CVEs by library name for O(1) lookup. Matching follows a two-tier policy:

- **Tier 1 — Name Match:** Any library name in the vulnerability DB raises a finding. This is the primary criterion because the dataset's version ranges provably contradict ground-truth labels (exact version matching yields 0% detection; name matching yields 173/176).
- **Tier 2 — Version Range:** The version_affected() function checks if the version falls within the CVE's documented range. Supports exact strings, comparators, and range pairs (with auto-normalization of 39 reversed ranges). Reported as a confidence tier ("name+version-range" vs "library-name match"), never as a filter.

3.4 License Compliance Engine

A deterministic, rule-table-driven checker with distribution-scope awareness:

Condition	App Model	Result
Viral copyleft (GPL/AGPL)	Proprietary	CONFLICT
Viral copyleft	Internal-only	ADVISORY (no distribution)
Unknown/undeclared license	Any	UNKNOWN (legal review)
All other licenses	Any	OK

3.5 Findings Generation & Classification

The findings module generates findings using the official taxonomy:

VULNERABLE_DEPENDENCY, TRANSITIVE_VULNERABILITY, LICENSE_CONFLICT, TRANSITIVE_LICENSE_CONFLICT, LICENSE_UNKNOWN, UNMAINTAINED, and LICENSE_ADVISORY. A dependency can trigger multiple findings; a single `primary_type` is derived using this priority: LICENSE_CONFLICT > VULN_CONFIRMED > VULN_NAME_MATCH > LICENSE_UNKNOWN > UNMAINTAINED.

The UNMAINTAINED cutoff uses `REFERENCE_DATE = 2026-04-01` minus 730 days (= 2024-04-01), not `date.today()`, so results are reproducible and consistent with the calendar-anchored ground truth.

3.6 ML Feature Engineering & Scoring

12 features per dependency, derived from core outputs only (no label leakage):

Feature	Source	Description
cvss_score	vuln_match	Max CVSS across name-matched CVEs
has_patch	vuln_match	Any matched CVE has patch_available
version_in_range	vuln_match	Version falls in affected range
num_cves	vuln_match	Count of name-matched CVEs
dependency_depth	graph	Shortest hops from app (0 = direct)

days_since_update	SBOM	Staleness signal
license_risk_tier	license engine	0=LOW / 1=MED / 2=HIGH / 3=CRIT
license_conflict	license engine	Viral copyleft in proprietary app
license_unknown	license engine	Undeclared license
num_paths	graph	Diamond signal (compound exposure)
is_shared_across_apps	SBOM	Library in 2+ applications
app_criticality	applications	Application criticality level

The baseline formula uses hand-tuned weights: name_match=0.45, cvss=0.15, license_conflict=0.70 (max, not additive), unmaintained=0.50, plus small boosts for multi-path (+0.05) and shared (+0.03). Threshold 0.40 → risky.

3.7 Composite Report & Severity

The composite module merges findings with risk scores and deterministic severity classes. Multi-path findings receive a compound uplift: +5% per extra path, capped +15%. Per-app risk score = mean of top-10 finding scores. Applications ranked by this score.

Severity rules:

- **Vulnerability:** CVE severity field of highest-CVSS matched CVE
- **License Conflict:** CRITICAL for AGPL/GPL-3.0; HIGH for others
- **License Unknown:** MEDIUM (legal review needed)
- **Unmaintained:** HIGH if > 3 years stale; MEDIUM if 2-3 years

3.8 Diamond/Compound Exposure

When the same library is reachable by 2+ paths (a diamond), each path is a separate exposure with its own remediation surface. The system enforces this at every level: all_simple_paths returns ALL paths; findings carry num_paths and compound_exposure flags; risk scores receive compound uplift; the chain viewer draws both branches converging. 105 of the current findings are multi-path/compound.

4. User Interface Design

4.1 Design Philosophy

The dashboard is a single-page, zero-external-asset web application (Flask + Jinja2) with a premium dark-mode aesthetic optimized for SOC environments. All CSS, scripts, fonts, and logos are inline/embedded — the page works fully offline. A comprehensive @media print stylesheet converts to a legible light-mode layout for PDF export.

Visual System

Element	Treatment
Background	Dark (#08090c) with radial brand-red and blue gradient glows
Glassmorphism	Sticky navbar with backdrop-filter: blur(14px) saturate(140%)
Severity colors	CRITICAL=#ff3b4e, HIGH=#ff8f3d, MEDIUM=#ffc531, LOW=#35d07f
Brand accent	Societe Generale red (#E60028) for key UI elements
Typography	Inter/system-ui, monospace for IDs and code; tabular-nums for alignment
Animations	Entry rise effects, count-up numbers, pulse indicators, hover lifts

4.2 Dashboard Components

The interface follows a progressive disclosure pattern: Overview → Drill-down.

- **Navigation Header:** Sticky glassmorphic bar with SG logo (base64-embedded), title hierarchy, Team Zenith badge, Export HTML and Print/PDF buttons. A 3px gradient topline provides brand anchoring.
- **KPI Tiles:** Four glassmorphic tiles with animated count-up numbers (cubic ease-out, 900ms): Applications, Dependencies Scanned, Critical Findings (with pulsing red glow when > 0), and Compound Exposures.
- **Risk Overview Charts:** SVG donut chart (severity distribution) built in vanilla JS with animated arcs + legend. Horizontal bar chart ranking top-10 applications by risk score with gradient fills.

- **Application Scorecards:** Responsive grid of clickable cards showing per-app risk profiles: name, score, criticality/license chips, animated progress bar, finding counts. Click applies an app filter to the findings table. Active card receives red border with glow.

4.3 Findings Table

The core data view is a full-width table with sticky header (blurred background) and these columns:

Column	Content
#	Sequential rank number (monospace)
Score	0-100 risk score + gradient minibar + compound uplift %
Severity	Color-coded pill badge with pulsing dot for CRITICAL
App / Type	Application ID; finding type with advisory flag
Library	Name@version, diamond indicator (◆ N paths), ◆ view chain button
Dependency Path	Full chain: app → lib1 → lib2 → flagged-lib (red bold leaf)
Evidence	CVE details (ID, CVSS, patch status, confidence tier), license info, or staleness data
AI Narrative	Blue-bordered callout with remediation prose + source label (when cached)

4.4 Dependency Chain Viewer

The "◆ view chain" button opens a modal overlay with an offline SVG visualization of the dependency chain. The graph renderer is built in vanilla JavaScript with no dependencies. Nodes are arranged in columns by depth level: blue boxes for applications, dark boxes for intermediaries, red boxes for flagged libraries. Edges are drawn as cubic Bezier curves. Diamond dependencies show both branches converging on the shared leaf. The modal supports ESC-to-close and click-outside-to-dismiss.

4.5 Filtering, Export & Accessibility

- **Filtering:** Application and finding-type dropdown filters with auto-submit. Filtered views are URL-shareable via GET parameters. Clear-filters button when active.
- **Export:** HTML export produces a fully self-contained file (all CSS inline, logos as data URIs). Print/PDF uses @media print stylesheet: white background, dark text, no animations.
- **Responsive & Accessibility:** 820px breakpoint collapses charts to single column. prefers-reduced-motion disables all animations. Semantic HTML, keyboard navigation (ESC for modal), tabular numerics for aligned columns.

5. Validation & Testing

5.1 Edge Case Testing (Phase 5)

Five hand-crafted scenarios run through the real deterministic core (isolated from training data, self-eval, and ML scoring):

#	Scenario	Result
1	Critical CVE with patch available — flagged but patch noted in evidence	PASS
2	Unmaintained 2+ years, zero CVEs — still flagged UNMAINTAINED	PASS
3	GPL in internal-only app → ADVISORY; in proprietary → CONFLICT	PASS
4	Same vulnerable lib in two apps — two findings, not deduplicated	PASS
5	Genuine diamond — one finding, num_paths=2, both chains reported	PASS

5.2 Unit Tests

Graph unit tests use a hand-built 6-node fixture (APP-X with libA→libC→libV and libB→libV diamond + clean libD). Tests verify: exact path set enumeration, hop

counts, clean-lib single-path correctness, diamond non-deduplication, and version matching (exact, comparator, near-miss). All tests pass.

6. Project Layout

Path	Purpose
src/config.py	DATA_DIR resolution (env override → sample_data/)
src/ingestion.py	Stdlib loader + schema/cross-reference validation
src/graph/builder.py	DiGraph construction with per-app subgraph isolation
src/graph/traversal.py	Exhaustive path enumeration (DFS) + depth_of
src/vuln_match.py	Name-match CVE policy + version-range confidence tier
src/license/engine.py	Distribution-scope-aware license decision table
src/findings.py	Findings generation + per-dep primary classification
src/ml/features.py	12 engineered features per dependency
src/ml/train.py	Baseline formula, GBDT, honest holdout model selection
src/report/composite.py	Ranked per-app report, severity, compound uplift
src/llm/narrative.py	Cache-first narratives, Ollama + template fallback
dashboard/app.py	Flask app, CORS, HTML export, health endpoint
dashboard/templates/dashboard.html	Dark-mode dashboard, charts, chain viewer
self_eval/evaluate.py	Self-eval against ground truth (core only)
tests/	test_graph.py (unit) + edge_cases.py (Phase 5)
reports/	Generated: findings, scores, report, narratives, edge_cases.md

— *End of Document* —

Zenith · Software Supply Chain Risk Scorer · Societe Generale iHackMyPlace · July
2026